

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS
Carrera de Ingeniería Informática
INF781 — SEGURIDAD DE SOFTWARE
GUÍA DE LABORATORIO N° 3

Registro y Login Seguro con Hashing de Contraseñas

Proyecto: tasks-app | bcrypt vs MD5/SHA1 | Auth::attempt() | Middleware auth

Materia:	INF781 Seguridad de Software	Semestre:	7° Semestre
Docente:	M.Sc. Huáscar F. Gonzales G.	Gestión:	2025
Duración:	5 horas académicas	Modalidad:	Presencial
Guía N°:	3 de 15	Proyecto:	tasks-app

 **COMPETENCIA:** Implementa un sistema de autenticación seguro en Laravel 12 aplicando hashing adaptativo con bcrypt, reglas estrictas de contraseña mediante Password rules, y protección de rutas con middleware auth; comparando científicamente por qué MD5 y SHA1 son inseguros para el almacenamiento de contraseñas.

Relación con Guías Anteriores

Esta guía construye sobre las Guías 1 y 2. De la Guía 1 se necesita el proyecto tasks-app con PostgreSQL y la tabla users con UUID. De la Guía 2 se reutiliza el patrón de Form Requests y el middleware de sanitización. En esta guía se implementa el sistema completo de autenticación que protegerá todas las rutas de las guías siguientes.

 **PREREQUISITO:** Guías 1 y 2 completadas. Tabla users creada con UUID. El UserSeeder de la Guía 2 puede eliminarse ya que ahora se creará un registro real. Verificar: php artisan migrate:status

Objetivos Específicos

- Entender por qué MD5 y SHA-1 son inseguros para contraseñas mediante demostración práctica medible.
- Configurar el costo de bcrypt en config/hashing.php y comprender su impacto en seguridad vs rendimiento.
- Implementar RegisterController con Hash::make() y reglas Password avanzadas.
- Implementar LoginController con Auth::attempt() y protección contra ataques de fuerza bruta (throttle).
- Proteger rutas del sistema tasks-app con el middleware auth de Laravel.
- Configurar Argon2id como alternativa moderna a bcrypt.

■ 1. Marco Teórico: Hashing de Contraseñas

1.1 Por qué NO usar MD5 ni SHA-1 para Contraseñas

MD5 y SHA-1 son funciones de hash criptográfico de propósito general. Fueron diseñadas para ser rápidas, lo cual es exactamente el problema cuando se usan para contraseñas.

Característica	MD5	SHA-1	bcrypt (cost=12)
Velocidad	~11 GB/hash/s (GPU)	~7 GB/hash/s (GPU)	~3 hash/s por núcleo
Salt integrado	No (debe agregarse manual)	No (debe agregarse manual)	Sí (generado automáticamente)
Colisiones conocidas	Sí (2004)	Sí (2017 - SHAttered)	No (diseño diferente)
Hashes en 1 hora (GPU)	~40 billones	~25 billones	~10.800
Recomendado NIST	No (obsoleto)	No (obsoleto)	Sí (hasta 2030+)
Rainbow tables	Ampliamente disponibles	Ampliamente disponibles	Ineficaces (salt único)



Principio Clave: Una función de hash para contraseñas debe ser LENTA de forma intencional (adaptativa). bcrypt, Argon2 y scrypt están diseñados específicamente para esto. Cuanto más tiempo tarda en calcular un hash, más tiempo le toma al atacante probar millones de combinaciones.

1.2 bcrypt: Diseño y Factor de Costo

bcrypt fue diseñado en 1999 por Niels Provos y David Mazières específicamente para el hashing de contraseñas. Su característica más importante es el factor de costo (cost factor): un parámetro que controla cuántas iteraciones realiza el algoritmo.

Cost	Iteraciones (2^{cost})	Tiempo aprox. / hash (CPU moderno)
10	1.024	~100 ms — mínimo aceptable
11	2.048	~200 ms
12	4.096	~400 ms — recomendado producción (2025)
13	8.192	~800 ms — alta seguridad
14	16.384	~1.600 ms — muy alta seguridad

La fórmula es: iteraciones = 2^{cost} . Con cost=12 se realizan 4.096 iteraciones internas. Un atacante con una GPU que puede calcular 10.000 millones de hashes MD5 por segundo, solo puede calcular ~3 hashes bcrypt/cost=12 por segundo en el mismo hardware.

1.3 Argon2id: El Sucesor Moderno

Argon2id ganó la Password Hashing Competition en 2015 y es recomendado por OWASP para nuevas aplicaciones. A diferencia de bcrypt, resiste ataques con hardware especializado (ASICs/FPGAs) porque usa memoria RAM como parte del algoritmo.

- memory: cantidad de RAM que consume (en KB) — dificulta ataques con hardware especializado
- time: número de iteraciones — equivalente al cost de bcrypt
- threads: paralelismo — Laravel usa 1 por defecto

1.4 Password Rules en Laravel 12

Laravel 12 incluye la clase `Illuminate\Validation\Rules\Password` que permite definir reglas de contraseña encadenadas de forma expresiva y legible:

```
use Illuminate\Validation\Rules\Password;

Password::min(8)           // mínimo 8 caracteres

->mixedCase()              // al menos una mayúscula y una minúscula

->numbers()                // al menos un número

->symbols()                 // al menos un símbolo (@#$%!...)

->uncompromised()          // verifica contra HaveIBeenPwned (opcional)
```



Nota sobre `uncompromised()`: Este método verifica la contraseña contra la base de datos de Have I Been Pwned usando k-anonymity (envía solo los primeros 5 caracteres del hash SHA-1, nunca la contraseña completa). Es opcional pero recomendado en producción. Requiere conexión a internet.

■ 2. Procedimiento: Implementación del Sistema de Autenticación

Script de Demostración: MD5 vs SHA-1 vs bcrypt

Antes de implementar, realizamos la demostración práctica que evidencia por qué MD5 y SHA-1 son inseguros. Crea el archivo `scripts/hash_comparison.php` en la raíz del proyecto:

```
<?php

/**
 * INF781 - Seguridad de Software
 * Demostración: Comparación de algoritmos de hashing
 * Ejecutar: php scripts/hash_comparison.php
 */

// — Contraseñas de prueba —————
$passwords = [
    'password123',          // contraseña débil común
    'Admin@2025!',          // contraseña moderada
    'T@skS3cur3_Lab!2025', // contraseña fuerte
];

echo str_repeat('=', 70) . PHP_EOL;
echo ' INF781 — Comparación de Algoritmos de Hashing' . PHP_EOL;
echo str_repeat('=', 70) . PHP_EOL . PHP_EOL;

foreach ($passwords as $pwd) {
    echo "Contraseña: {$pwd}" . PHP_EOL;
    echo str_repeat('-', 70) . PHP_EOL;

    // — MD5 (INSEGURO) —————

    $t = microtime(true);

    $hash = md5($pwd);

    $elapsed = (microtime(true) - $t) * 1000;

    printf(" MD5      : %s\n", $hash);

    printf(" Tiempo  : %.6f ms\n", $elapsed);
}
```

Paso 1

```
    printf(" Longitud: %d chars | INSEGURO: sin salt, colisiones conocidas\n",
strlen($hash));

    echo PHP_EOL;

// — SHA-1 (INSEGURO) —————

    $t = microtime(true);

    $hash = sha1($pwd);

    $elapsed = (microtime(true) - $t) * 1000;

    printf("  SHA-1   : %s\n", $hash);

    printf(" Tiempo  : %.6f ms\n", $elapsed);

    printf(" Longitud: %d chars | INSEGURO: SHattered (2017), sin salt\n",
strlen($hash));

    echo PHP_EOL;

// — SHA-256 (mejor, pero aún NO recomendado para contraseñas) —

    $t = microtime(true);

    $hash = hash('sha256', $pwd);

    $elapsed = (microtime(true) - $t) * 1000;

    printf(" SHA-256: %s\n", $hash);

    printf(" Tiempo  : %.6f ms\n", $elapsed);

    printf(" Longitud: %d chars | Mejor, pero sigue siendo rápido = vulnerable\n",
strlen($hash));

    echo PHP_EOL;

// — bcrypt cost=10 —————

    $t = microtime(true);

    $hash = password_hash($pwd, PASSWORD_BCRYPT, ['cost' => 10]);

    $elapsed = (microtime(true) - $t) * 1000;

    printf(" bcrypt/10: %s\n", substr($hash, 0, 30) . '...');

    printf(" Tiempo  : %.2f ms | SEGURO: salt automático incluido\n", $elapsed);

    printf(" Salt    : %s\n", substr($hash, 7, 22));

    echo PHP_EOL;

// — bcrypt cost=12 —————

    $t = microtime(true);

    $hash = password_hash($pwd, PASSWORD_BCRYPT, ['cost' => 12]);
```

```

$elapsed = (microtime(true) - $t) * 1000;

printf("  bcrypt/12: %s\n", substr($hash, 0, 30) . '...');

printf("  Tiempo : %.2f ms | SEGURO: recomendado producción\n", $elapsed);

echo PHP_EOL;

// — Argon2id —————

if (defined('PASSWORD_ARGON2ID')) {

    $t = microtime(true);

    $hash = password_hash($pwd, PASSWORD_ARGON2ID, [

        'memory_cost' => 65536,    // 64 MB

        'time_cost'    => 4,

        'threads'      => 1,

    ]);

    $elapsed = (microtime(true) - $t) * 1000;

    printf("  Argon2id : %s\n", substr($hash, 0, 40) . '...');

    printf("  Tiempo   : %.2f ms | MUY SEGURO: resiste ASICs\n", $elapsed);

    echo PHP_EOL;

}

// — Verificación de hash —————

echo '  VERIFICACIÓN bcrypt:' . PHP_EOL;

$storedHash = password_hash($pwd, PASSWORD_BCRYPT, ['cost' => 12]);

$correct     = password_verify($pwd, $storedHash);

$incorrect   = password_verify('wrong', $storedHash);

printf("  password_verify(correcto) : %s\n", $correct ? 'true ✓' : 'false X');

printf("  password_verify(incorrecto): %s\n", $incorrect ? 'true X' : 'false ✓');

echo PHP_EOL . str_repeat('=', 70) . PHP_EOL . PHP_EOL;

}

// — Demostración: mismo input → mismo MD5 (predecible) —————

echo 'PROBLEMA DE MD5: Mismo input SIEMPRE produce mismo output' . PHP_EOL;

echo '  Sin salt: todos los usuarios con "password" tienen el mismo hash' . PHP_EOL;

echo '  MD5("password") = ' . md5('password') . PHP_EOL;

echo '  (Este hash aparece en TODAS las rainbow tables)' . PHP_EOL;

```

```

echo PHP_EOL;

// — Demostración: bcrypt genera salt único por hash —————
echo 'VENTAJA bcrypt: Mismo input → hashes DIFERENTES (salt único)' . PHP_EOL;

$p = 'password';

echo '  hash1: ' . password_hash($p, PASSWORD_BCRYPT) . PHP_EOL;
echo '  hash2: ' . password_hash($p, PASSWORD_BCRYPT) . PHP_EOL;
echo '  (Son distintos, pero password_verify() funciona con ambos)' . PHP_EOL;

```

```

# Ejecutar el script de demostración
php scripts/hash_comparison.php

# Resultado esperado (tiempos aproximados):
# MD5      : 0.000032 ms ← casi instantáneo (PELIGROSO)
# SHA-1    : 0.000041 ms ← casi instantáneo (PELIGROSO)
# SHA-256: 0.000058 ms ← casi instantáneo (vulnerable)
# bcrypt/10: ~85 ms     ← intencionalmente lento (SEGURO)
# bcrypt/12: ~340 ms    ← más lento (MÁS SEGURO)
# Argon2id : ~120 ms    ← memoria intensivo (MUY SEGURO)

```

 **Interpretación:** La diferencia de ~340ms (bcrypt) vs 0.000032ms (MD5) parece pequeña para un usuario legítimo. Pero para un atacante que intenta 1.000.000 de contraseñas: MD5 tarda 32ms totales, bcrypt/12 tarda ~94 HORAS. Eso es la seguridad adaptativa.

Configuración de hashing (opcional)

Laravel 13 utiliza bcrypt con 12 rounds por defecto sin necesidad de configuración alguna. El archivo config/hashing.php no existe en un proyecto nuevo y no es necesario crearlo a menos que se requiera modificar explícitamente los valores predeterminados.

Si se desea personalizar el costo de bcrypt, primero se publica el archivo de configuración:

```
php artisan config:publish hashing
```

Y luego se ajustan los valores en config/hashing.php. Alternativamente, se puede definir la variable BCRYPT_ROUNDS directamente en el archivo .env sin necesidad de publicar nada.

```

<?php

// config/hashing.php

return [

```

**Pas
o 2**

```

/*
| DRIVER DE HASHING POR DEFECTO
| Opciones: 'bcrypt', 'argon', 'argon2id'
| bcrypt es el más compatible; argon2id es el más moderno.
*/

'driver' => env('HASH_DRIVER', 'bcrypt'),

/*
| CONFIGURACIÓN DE BCrypt
| Cost recomendado para 2025: 12
| Incrementar si el hardware del servidor lo permite.
| Regla práctica: el hash debe tardar ~300-500ms en el servidor.
*/

'bcrypt' => [
    'rounds' => env('BCRYPT_ROUNDS', 12),
],

/*
| CONFIGURACIÓN DE ARGON2 (argon / argon2id)
| memory: RAM en KB (65536 = 64 MB es el mínimo recomendado OWASP)
| time : iteraciones (mínimo 3 para Argon2id según OWASP)
| threads: siempre 1 en aplicaciones web (evitar contención)
*/

'argon' => [
    'memory' => env('ARGON_MEMORY', 65536),
    'time' => env('ARGON_TIME', 4),
    'threads' => env('ARGON_THREADS', 1),
],

];

```

Variables de entorno para hashing (opcional)

Laravel 13 configura el hashing automáticamente con valores seguros por defecto. No es necesario agregar estas variables al archivo `.env` a menos que se requiera modificar el comportamiento predeterminado:

`HASH_DRIVER=bcrypt`


```
BCRYPT_ROUNDS=12
HASH_VERIFY=true
```

Estos son precisamente los valores que Laravel ya utiliza internamente, por lo que solo tiene sentido declararlos si se desea cambiar alguno, por ejemplo, aumentar los rounds o cambiar el driver a argon2id.

```
# .env - Configuración de hashing

HASH_DRIVER=bcrypt

BCRYPT_ROUNDS=12

# Para Argon2id (opcional - descomentar si se cambia el driver):
# HASH_DRIVER=argon2id
# ARGON_MEMORY=65536
# ARGON_TIME=4
# ARGON_THREADS=1
```

Verifica la configuración en Tinker:

```
php artisan tinker

>>> config('hashing.driver');
// => 'bcrypt'

>>> config('hashing.bcrypt.rounds');
// => 12

// Medir el tiempo real de hashing con cost=12

>>> $t = microtime(true); Hash::make('test'); echo round((microtime(true)-$t)*1000,1).'ms';

// Debe mostrar entre 200ms y 500ms según el hardware

// Verificar que el hash resultante incluye el cost

>>> Hash::make('password');

// => '$2y$12$...' - el '12' confirma el costo configurado
```

Paso 3

Crear el Form Request de Registro

Creamos el Form Request con las reglas de contraseña avanzadas de Laravel 12:

```
php artisan make:request Auth\RegisterRequest
```

Edita app/Http/Requests/Auth/RegisterRequest.php:

```
<?php

namespace App\Http\Requests\Auth;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rules\Password;
use Illuminate\Contracts\Validation\ValidationRule;
use Illuminate\Support\Facades\Auth;

class RegisterRequest extends FormRequest
{
    public function authorize(): bool
    {
        // Solo usuarios no autenticados pueden registrarse
        return !Auth::check();
    }

    public function rules(): array
    {
        return [
            'name' => [
                'required',
                'string',
                'min:2',
                'max:100',
                'regex:/^[p{L}s\-\.\.]+$/u', // Solo letras, espacios, guiones y puntos
            ],
            'email' => [
                'required',
                'string',
                'email:rfc,dns',
            ],
        ];
    }
}
```

```

        'max:150',

        'unique:users,email', // No puede existir en la BD

    ],

    'password' => [

        'required',

        'string',

        // Encadenamiento de reglas de contraseña:

        Password::min(8)          // Mínimo 8 caracteres

            ->mixedCase()          // Al menos 1 mayúscula + 1 minúscula

            ->numbers()            // Al menos 1 número

            ->symbols()            // Al menos 1 símbolo (!@#$$%...)

            // ->uncompromised() // Verifica HaveIBeenPwned (producción)

        ,

        'confirmed', // Requiere campo password_confirmation

    ],

    // Este campo es validado automáticamente por 'confirmed'

    'password_confirmation' => ['required', 'string'],

];

}

public function messages(): array
{

    return [

        'name.required'      => 'El nombre completo es obligatorio.',

        'name.min'           => 'El nombre debe tener al menos :min caracteres.',

        'name.regex'         => 'El nombre solo puede contener letras, espacios y
guiones.',

        'email.required'     => 'El correo electrónico es obligatorio.',

        'email.email'        => 'El correo electrónico no tiene un formato válido.',

        'email.unique'       => 'Este correo ya está registrado. ¿Desea iniciar
sesión?',
    ];
}

```

```

        'password.required'      => 'La contraseña es obligatoria.',
        'password.min'          => 'La contraseña debe tener al menos :min
caracteres.',
        'password.mixed'       => 'La contraseña debe contener mayúsculas y
minúsculas.',
        'password.numbers'     => 'La contraseña debe contener al menos un
número.',
        'password.symbols'     => 'La contraseña debe contener al menos un símbolo
(@#$!...).',
        'password.confirmed'   => 'La confirmación de contraseña no coincide.',

        'password_confirmation.required' => 'Debe confirmar su contraseña.',

    ];
}

protected function prepareForValidation(): void
{
    $this->merge([

        'name'  => trim($this->name ?? ''),
        'email' => strtolower(trim($this->email ?? '')),

        // IMPORTANTE: NO sanitizar password con strip_tags/trim
        // Los símbolos son parte válida de la contraseña

    ]);
}
}

```

Crear el Form Request de Login

```
php artisan make:request Auth\LoginRequest
```

Edita app/Http/Requests/Auth/LoginRequest.php:

```

<?php

namespace App\Http\Requests\Auth;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;

```

**Pas
o 4**

```
use Illuminate\Support\Facades\Auth;

use Illuminate\Validation\ValidationException;

class LoginRequest extends FormRequest
{
    public function authorize(): bool
    {
        return !auth()->check();
    }

    public function rules(): array
    {
        return [
            'email' => ['required', 'string', 'email'],
            'password' => ['required', 'string'],
            // remember_me es opcional
            'remember' => ['boolean'],
        ];
    }

    public function messages(): array
    {
        return [
            'email.required' => 'El correo electrónico es obligatorio.',
            'email.email' => 'Ingresa un correo electrónico válido.',
            'password.required' => 'La contraseña es obligatoria.',
        ];
    }

    /**
     * Verificar el límite de intentos antes de procesar el login.
     * Protección contra ataques de fuerza bruta.
     */
    public function ensureIsNotRateLimited(): void
```

```
{  
  
    // Clave única por email + IP del cliente  
  
    $key = Str::transliterate(  
  
        Str::lower($this->email) . '|' . $this->ip()  
  
    );  
  
  
  
    if (!RateLimiter::tooManyAttempts($key, 5)) {  
  
        return; // Menos de 5 intentos, continuar  
  
    }  
  
  
  
    $seconds = RateLimiter::availableIn($key);  
  
  
  
    throw ValidationException::withMessages([  
  
        'email' => trans('auth.throttle', [  
  
            'seconds' => $seconds,  
  
            'minutes' => ceil($seconds / 60),  
  
        ]),  
  
    ]);  
  
}  
  
  
  
public function incrementRateLimiter(): void  
  
{  
  
    $key = Str::transliterate(  
  
        Str::lower($this->email) . '|' . $this->ip()  
  
    );  
  
    RateLimiter::hit($key, 300); // Bloqueo de 5 minutos  
  
}  
  
  
  
public function clearRateLimiter(): void  
  
{  
  
    $key = Str::transliterate(  
  
        Str::lower($this->email) . '|' . $this->ip()  
  
    );  
  
    RateLimiter::clear($key);  
  
}
```

```
}  
  
}
```

Crear los Controladores de Autenticación

```
php artisan make:controller Auth/RegisterController
```

```
php artisan make:controller Auth/LoginController
```

Edita app/Http/Controllers/Auth/RegisterController.php:

```
<?php  
  
namespace App\Http\Controllers\Auth;  
  
use App\Http\Controllers\Controller;  
use App\Http\Requests\Auth\RegisterRequest;  
use App\Models\User;  
use Illuminate\Auth\Events\Registered;  
use Illuminate\Http\RedirectResponse;  
use Illuminate\Support\Facades\Auth;  
use Illuminate\Support\Facades\Hash;  
use Illuminate\View\View;  
  
class RegisterController extends Controller  
{  
    public function create(): View  
    {  
        return view('auth.register');  
    }  
  
    public function store(RegisterRequest $request): RedirectResponse  
    {  
        $validated = $request->validated();  
  
        // Hash::make() usa bcrypt con el cost configurado en hashing.php  
        // NUNCA almacenar la contraseña en texto plano  
        // NUNCA usar md5() o sha1() aquí
```

**Pas
o 5**

```

        $user = User::create([
            'name'      => $validated['name'],
            'email'     => $validated['email'],
            'password' => Hash::make($validated['password']),
        ]);

        // Disparar evento Registered (permite verificación de email)
        event(new Registered($user));

        // Autenticar al usuario inmediatamente después del registro
        Auth::login($user);

        return redirect()->route('tasks.index')
            ->with('success', '¡Cuenta creada exitosamente! Bienvenido/a, ' . $user->name);
    }
}

```

Edita app/Http/Controllers/Auth/LoginController.php:

```

<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class LoginController extends Controller
{
    public function create(): View
    {

```



```
        return view('auth.login');
    }

    public function store(LoginRequest $request): RedirectResponse
    {
        // Verificar rate limiting ANTES de intentar la autenticación
        $request->ensureIsNotRateLimited();

        // Auth::attempt() internamente usa Hash::check() (password_verify)
        // Compara la contraseña ingresada contra el hash bcrypt en la BD
        $credentials = $request->only('email', 'password');
        $remember     = $request->boolean('remember');

        if (!Auth::attempt($credentials, $remember)) {
            // Login fallido: incrementar contador de intentos
            $request->incrementRateLimiter();

            // IMPORTANTE: mensaje genérico (no revelar si email existe)
            throw ValidationException::withMessages([
                'email' => 'Las credenciales proporcionadas no son correctas.',
            ]);
        }

        // Login exitoso: limpiar rate limiter y regenerar sesión
        $request->clearRateLimiter();

        // Regenerar el ID de sesión previene Session Fixation Attack
        $request->session()->regenerate();

        return redirect()->intended(route('tasks.index'))
            ->with('success', 'Sesión iniciada correctamente.');
```

```
    }

    public function destroy(Request $request): RedirectResponse
```

```

{
    Auth::logout();

    // Invalida la sesión completamente
    $request->session()->invalidate();

    // Regenera el token CSRF (previene CSRF post-logout)
    $request->session()->regenerateToken();

    return redirect()->route('login')
        ->with('success', 'Has cerrado sesión correctamente.');
```

Actualizar Rutas y Proteger con Middleware auth

Actualiza routes/web.php para incluir las rutas de autenticación y proteger las rutas de tasks:

```

<?php

// routes/web.php
use App\Http\Controllers\Auth\LoginController;
use App\Http\Controllers\Auth\RegisterController;
use App\Http\Controllers\TaskController;
use Illuminate\Support\Facades\Route;
use Illuminate\Support\Facades\Auth;

// — Ruta raíz —————
Route::get('/', function () {
    return Auth::check()
        ? redirect()->route('tasks.index')
        : redirect()->route('login');
});

// — Rutas de autenticación (solo para invitados) —————
Route::middleware('guest')->group(function () {
```

**Pas
o 6**

```
// Registro

Route::get('/register', [RegisterController::class, 'create'])->name('register');

Route::post('/register', [RegisterController::class, 'store']);

// Login

Route::get('/login', [LoginController::class, 'create'])->name('login');

Route::post('/login', [LoginController::class, 'store']);

});

// — Logout (solo usuarios autenticados) —————

Route::middleware('auth')->group(function () {

    Route::post('/logout', [LoginController::class, 'destroy'])->name('logout');

// — Rutas protegidas de tasks —————

// El middleware 'auth' redirige a /login si no hay sesión activa

Route::resource('tasks', TaskController::class);

});
```

 **Cómo funciona el middleware 'auth':** Cuando un usuario no autenticado intenta acceder a /tasks, Laravel intercepta la petición y redirige a /login. Después del login exitoso, Auth::attempt() restaura la URL original via redirect()->intended().

Actualiza TaskController para usar el usuario autenticado (eliminar el UUID hardcodeado de la Guía 2):

```
// Incluya en la sección use

use Illuminate\Support\Facades\Auth;

// En TaskController@store — reemplazar la línea del user_id hardcodeado:

// ANTES (Guía 2 — temporal):

// $validated['user_id'] = '00000000-0000-0000-0000-000000000001';

// AHORA (Guía 3 — correcto):

$validated['user_id'] = Auth::id();
```

```
// También en index(), filtrar solo las tareas del usuario autenticado:

public function index(): View
{
    $tasks = Task::where('user_id', Auth::id())
        ->latest()
        ->paginate(10);

    return view('tasks.index', compact('tasks'));
}
```

Crear las Vistas Blade de Autenticación

Crea resources/views/auth/register.blade.php:

```
@extends('layouts.app')

@section('title', 'Crear Cuenta')

@section('content')
<div style="max-width:480px; margin:0 auto;">
    <h2 style="color:#025043; border-bottom:3px solid #025043;
        padding-bottom:8px;">Crear Cuenta Nueva</h2>

    @if($errors->any())
        <div class="alert-error">
            <strong>Corrija los siguientes errores:</strong>
            <ul>@foreach($errors->all() as $e)<li>{{ $e }}</li>@endforeach</ul>
        </div>
    @endif

    <form action="{{ route('register') }}" method="POST">
        @csrf

        <div class="form-group">
            <label for="name">Nombre Completo *</label>
            <input type="text" id="name" name="name"
                value="{{ old('name') }}"
                placeholder="Ej: Juan Pérez López">
        </div>
    </form>
</div>
```

**Pas
o 7**

```

        autocomplete="name" required>

        @error('name')<span class="error-msg">{{ $message }}</span>@enderror
    </div>

    <div class="form-group">
        <label for="email">Correo Electrónico *</label>

        <input type="email" id="email" name="email"

            value="{{ old('email') }}"

            placeholder="correo@ejemplo.com"

            autocomplete="email" required>

        @error('email')<span class="error-msg">{{ $message }}</span>@enderror
    </div>

    <div class="form-group">
        <label for="password">Contraseña *</label>

        <input type="password" id="password" name="password"

            placeholder="Mínimo 8 chars, mayús., número, símbolo"

            autocomplete="new-password" required>

        <small style="color:#64748b;">
            Debe contener: mayúsculas, minúsculas, números y símbolos (@#$.!)
        </small>

        @error('password')<span class="error-msg">{{ $message }}</span>@enderror
    </div>

    <div class="form-group">
        <label for="password_confirmation">Confirmar Contraseña *</label>

        <input type="password" id="password_confirmation"

            name="password_confirmation"

            placeholder="Repita la contraseña"

            autocomplete="new-password" required>

        @error('password_confirmation')

            <span class="error-msg">{{ $message }}</span>

        @enderror
    </div>

```

```

--}}

{{!-- Indicador visual de fortaleza (solo UI, la validación real es server-side)
--}}

<div style="background:#f0faf7; border:1px solid #025043;

padding:12px; border-radius:6px; margin-bottom:16px;

font-size:0.875rem;">

<strong style="color:#025043;">Requisitos de contraseña:</strong>

<ul style="margin:4px 0; padding-left:20px;">

<li>Mínimo 8 caracteres</li>

<li>Al menos una letra mayúscula (A-Z)</li>

<li>Al menos una letra minúscula (a-z)</li>

<li>Al menos un número (0-9)</li>

<li>Al menos un símbolo (!@#$$%^&*)</li>

</ul>

</div>

<button type="submit" class="btn btn-primary"

style="width:100%;">Crear Cuenta</button>

</form>

<p style="text-align:center; margin-top:16px;">

¿Ya tienes cuenta?

<a href="{{ route('login') }}">Iniciar Sesión</a>

</p>

</div>

@endsection

```

Crea resources/views/auth/login.blade.php:

```

@extends('layouts.app')

@section('title', 'Iniciar Sesión')

@section('content')

<div style="max-width:480px; margin:0 auto;">

<h2 style="color:#025043; border-bottom:3px solid #025043;

padding-bottom:8px;">Iniciar Sesión</h2>

```

```
@if($errors->any())

    <div class="alert-error">

        {{-- Mensaje genérico: no revelar si email existe o no --}}

        @foreach($errors->all() as $e)<p>{{ $e }}</p>@endforeach

    </div>

@endif

<form action="{{ route('login') }}" method="POST">

    @csrf

    <div class="form-group">

        <label for="email">Correo Electrónico</label>

        <input type="email" id="email" name="email"

            value="{{ old('email') }}"

            autocomplete="email" autofocus required>

        @error('email')<span class="error-msg">{{ $message }}</span>@enderror

    </div>

    <div class="form-group">

        <label for="password">Contraseña</label>

        <input type="password" id="password" name="password"

            autocomplete="current-password" required>

        @error('password')<span class="error-msg">{{ $message }}</span>@enderror

    </div>

    <div class="form-group" style="display:flex; align-items:center; gap:8px;">

        <input type="checkbox" id="remember" name="remember" value="1"

            {{ old('remember') ? 'checked' : '' }}>

        <label for="remember" style="margin:0; font-weight:normal;">

            Recordarme en este dispositivo</label>

    </div>

    <button type="submit" class="btn btn-primary">
```

```

        style="width:100%;">Iniciar Sesión</button>

</form>

<p style="text-align:center; margin-top:16px;">
    ¿No tienes cuenta?

    <a href="{{ route('register') }}">Crear cuenta nueva</a>

</p>

</div>

@endsection

```

Actualiza el layout `resources/views/layouts/app.blade.php` para mostrar el usuario autenticado y el botón de logout:

```

{{-- En el <header> del layout, reemplaza la sección de navegación: --}}

<header style="border-bottom:3px solid #025043; padding-bottom:12px; margin-
bottom:24px;">

    <div style="display:flex; justify-content:space-between; align-items:center;">

        <h1 style="color:#025043; margin:0;">

            <a href="{{ route('tasks.index') }}" style="text-
decoration:none;color:inherit;">

                Tasks App</a>

        </h1>

    </div>

    @auth

        {{-- @auth solo muestra contenido a usuarios autenticados --}}

        <span style="color:#64748b; margin-right:12px;">

            Hola, {{ auth()->user()->name }}</span>

        <a href="{{ route('tasks.index') }}">Mis Tareas</a> |

        <a href="{{ route('tasks.create') }}">Nueva Tarea</a> |

        <form action="{{ route('logout') }}" method="POST"

            style="display:inline;">

            @csrf

            <button type="submit"

                style="background:none;border:none;color:#dc2626;

                cursor:pointer;font-size:1rem;">

                Cerrar Sesión</button>

        </form>

```



```

        @else

        {{-- @else muestra a visitantes no autenticados --}}

        <a href="{{ route('login') }}">Iniciar Sesión</a> |

        <a href="{{ route('register') }}">Registrarse</a>

        @endauth

    </div>

</div>

</header>

```

Actualiza el app/Models/User.php

use Illuminate\Database\Eloquent\Concerns\HasUuids;

```

class User extends Authenticatable
{
    use HasUuids; // ← Esto genera el UUID automáticamente al crear

    // ... resto del modelo
}

```

Pruebas del Sistema de Autenticación

Realiza las siguientes pruebas en orden para verificar que el sistema funciona correctamente:

Prueba A — Registro con contraseña débil

```

# En el navegador: http://localhost:8000/register

# Intentar registrarse con:

#   Nombre: Juan

#   Email: juan@test.com

#   Contraseña: 12345678

#   Confirmación: 12345678

# Resultado esperado — Errores de validación:

#   'La contraseña debe contener mayúsculas y minúsculas.'

#   'La contraseña debe contener al menos un número.'    (ya cumple)

#   'La contraseña debe contener al menos un símbolo (@#$!...).'

```

Prueba B — Registro exitoso

```

# Registrarse con datos válidos:

#   Nombre: Ana García

```

**Pas
o 8**

```
# Email: ana@tasks-app.local
# Contraseña: Segur@2025!
# Confirmación: Segur@2025!

# Resultado esperado:
# Redirige a /tasks con mensaje de bienvenida
# La URL del navegador cambia a /tasks (ruta protegida)
# El navbar muestra: 'Hola, Ana García'

# Verificar en la BD que el hash es bcrypt:
psql -U tasks_user -d tasks_db -c \
    "SELECT name, email, LEFT(password,7) AS hash_prefix FROM users;"

# Resultado esperado:
# name      | email                | hash_prefix
# Ana García | ana@tasks-app.local | \2y$12$
# (el prefijo 2y$12$ confirma bcrypt con cost=12)
```

Prueba C — Protección de rutas con middleware auth

```
# Cerrar sesión, luego intentar acceder directamente a:
http://localhost:8000/tasks
http://localhost:8000/tasks/create

# Resultado esperado:
# Ambas URLs redirigen a http://localhost:8000/login
# Después del login, se redirige automáticamente a la URL original
```

Prueba D — Fuerza bruta (Rate Limiter)

```
# En el formulario de login, intentar 6 veces seguidas con
# contraseña incorrecta para el mismo email:
# Email: ana@tasks-app.local
# Contraseña: incorrecta (repetir 6 veces)

# Resultado esperado al 6° intento:
```

```
# 'Too many login attempts. Please try again in X seconds.'  
# (o el mensaje traducido si se configura lang/es/auth.php)  
  
# El rate limiter bloquea por 5 minutos (300 segundos)
```

■ 3. Capturas de Pantalla Esperadas

El estudiante debe documentar las siguientes 10 capturas en su reporte. La captura debe mostrar la URL completa del navegador o el prompt del terminal.

#	Momento	Resultado esperado	Dónde	Nota
1	Script hash_comparison.php	Terminal mostrando los tiempos: MD5 ~0.00003ms, bcrypt/12 ~300-400ms. La diferencia de velocidad es el mensaje central.	php script...	0-10
2	Formulario de registro	Página /register con todos los campos y la sección de requisitos de contraseña visible.	localhost/register	0-10
3	Error: contraseña débil	Mensajes de error en español bajo el campo password al intentar '12345678': falta mayúscula, falta símbolo.	localhost/register	0-10
4	Error: contraseñas no coinciden	Mensaje 'La confirmación de contraseña no coincide.' al poner confirmación diferente.	localhost/register	0-10
5	Error: email duplicado	Mensaje 'Este correo ya está registrado. ¿Desea iniciar sesión?' al intentar registrar un email existente.	localhost/register	0-10
6	Registro exitoso	Redirección a /tasks con el mensaje '¡Cuenta creada exitosamente! Bienvenido/a, [nombre]' y el navbar mostrando el nombre del usuario.	localhost/tasks	0-10
7	Hash bcrypt en PostgreSQL	Resultado del SELECT en psql mostrando el campo password con prefijo \$2y\$12\$ para el usuario registrado.	psql terminal	0-10
8	Middleware auth — redirección	Al acceder a /tasks sin sesión, el navegador redirige a /login (URL cambia visiblemente).	localhost/login	0-10
9	Login exitoso — redirect()->intended()	Después de intentar acceder a /tasks/create sin sesión y luego hacer login, la redirección lleva exactamente a /tasks/create.	localhost/tasks/create	0-10
10	Rate Limiter activado	Mensaje de bloqueo por demasiados intentos fallidos después del 6° intento.	localhost/login	0-10

Escala de Calificación

Rango	Calificación	Nota /100	Nivel de Logro
32–35	Excelente	91–100	Supera las expectativas
26–31	Bueno	74–90	Cumple completamente
18–25	Regular	51–73	Cumple parcialmente
0–17	Insuficiente	0–50	No cumple los mínimos